

15-418 Assignment 3: Parallel VLSI Wire Routing Report

Jason (jasont2) Catherine (catheri5)

October 8, 2026

1 Implementation Status

We first implemented a sequential correctness baseline before adding OpenMP parallelism. The starter code's compressed route representation was replaced with a keypoint representation: each wire stores up to five points, corresponding to the start point, up to three bends, and the end point. This matches the assignment checker format and makes route construction and validation easier to reason about.

At the start of timed computation, each input wire is assigned a simple legal route. If the endpoints share a row or column, the initial route is straight. Otherwise, the initial route is a horizontal-first one-bend path. The occupancy matrix is then constructed by adding each initial wire route.

The sequential correctness baseline visits every wire for each iteration. For a wire being reconsidered, the current route is first removed from the occupancy matrix. With fixed probability P , the program chooses a legal route uniformly at random. Otherwise, it evaluates every legal route with at most three bends using the sum of squared occupancy values that would result from adding the wire back, and chooses the lowest-cost route. The existing route is preserved unless a strictly cheaper candidate is found. The selected route is then added back into the occupancy matrix.

2 Route Generation and Cost Calculation

For non-collinear endpoints, route generation currently includes horizontal-first and vertical-first routes with at most two bends, plus exactly three-bend routes through each strictly interior point in the endpoint bounding box. Choosing strictly interior points for the three-bend case prevents those

routes from collapsing into duplicate one- or two-bend routes. Some at-most-two-bend routes do collapse to one-bend routes; consecutive duplicate keypoints are removed when constructing the wire.

Within-wires scoring removes the current wire before evaluating its candidates. Therefore each occupied cell on a W-mode candidate is scored using $(occupancy[y][x] + 1)^2$. Across-wires cannot generally remove the current wire during selection: another worker may be reading that occupancy cell while the current batch is still being scored. A-mode instead leaves the old route in the matrices, recognizes where a candidate overlaps it, and scores an overlapping cell as $occupancy[y][x]^2$ because the old wire’s removal and the candidate’s insertion cancel. Non-overlapping cells still use $(occupancy[y][x] + 1)^2$. Section 8 describes the resulting synchronization and geometry decisions in more detail.

3 Sequential Baseline Optimization

Table 1 summarizes the historical computation checkpoints on GHC for `few.wires.txt`, one thread, $P = 0.1$, five iterations, and batch size one. The early W-mode measurement provides context; the subsequent optimizations were first tested in the sequential A-mode baseline before being ported to W.

Implementation checkpoint	Mode	Compute (s)
Early vector-based candidate search	W	6.14
Generate and score immediately, without a candidate vector	A	≈ 5.89
Also specialize horizontal/vertical cost loops	A	3.17
Also maintain a contiguous column occupancy view	A	≈ 2.68

Table 1: Exploratory baseline progression, with times shown to three significant figures. The 5.89 s and 2.68 s values were reported approximately. The 3.17 s value is a representative run, with a reported median of approximately 3.15 s.

The recorded stages matter: eliminating the route vector alone left the A-mode baseline near 5.89 seconds, rather than immediately reaching 3.17. That larger reduction followed the axis-specific scoring loop; contiguous vertical access then reduced the reported time to about 2.68 seconds. From the 5.89-second A-mode checkpoint to 2.68 seconds, computation time fell by

approximately 54.5% (about $2.20\times$ speedup). These historical checkpoints were not all collected as a controlled median-of-three comparison, and W and A use different search paths, so we treat them as evidence of progress rather than a final benchmark comparison.

Before further within-wires tuning, we focused on the efficiency of the one-thread baseline. Originally, we stored all candidate routes in a vector before scoring them. We changed the sequential search to construct and fully score each candidate immediately, keeping only the current candidate and best route. This removes candidate-vector allocation, growth, and storage overhead without changing the exhaustive route set. The random branch constructs just the uniformly selected route without building the candidate list.

We also specialized the cost loop for horizontal and vertical segments. Horizontal scoring fixes a row and changes only the column index; vertical scoring fixes a column and changes only the row index. Initially, vertical access still crossed separately allocated rows. We subsequently added a transposed raw occupancy view, `occupancy_columns[x][y]`, alongside `occupancy[y][x]`. Horizontal segments now read contiguous rows and vertical segments read contiguous columns. Every wire removal and insertion updates both views. This trades additional memory and update work for better locality during the many candidate-scoring reads.

The transposed view is not a prefix-sum table or a cache of route costs. Every candidate is still evaluated completely, without cost reuse or early termination, and bends and the final endpoint are counted exactly once. The optimized traversal is now used by both modes. Both use indexed generation: A generates candidates sequentially within a batch, while W distributes candidate generation and scoring across threads as described below.

4 Within-Wires Parallelism

We keep iterations and the outer loop over wires sequential. After removing the current wire from both occupancy views, all candidates for that wire can be evaluated independently against the same read-only occupancy state. The chosen route is committed before moving to the next wire.

Our initial parallel implementation generated a candidate vector sequentially and parallelized only scoring. The current implementation instead maps an integer route index to one candidate with `route_from_index()`. Each worker constructs and scores its assigned candidates immediately, so

candidate generation is also parallel and the full route vector is no longer stored. The exact candidate count is available without constructing routes: for non-collinear endpoints it is

$$N_{\text{routes}} = d_x + d_y + 2(d_x - 1)(d_y - 1),$$

where d_x and d_y are absolute endpoint differences. Collinear endpoints have one candidate. This preserves a route-count-based sequential fallback.

Each thread maintains a private best route and cost. At the end of the parallel search, the main thread combines the per-thread results. This is a manual minimum reduction, analogous to accumulating partial sums independently and combining them once, rather than synchronizing after every candidate.

4.1 Tuning Methodology and Decisions

For early within-wires tuning, we temporarily measured only the iterative rerouting phase, excluding initial route placement and occupancy initialization. This narrower timing made it easier to isolate the effect of scheduling, chunk size, and threshold choices. Those older implementation measurements are not used for the current choices below. Current measurements use the assignment timing definition, where computation includes initial route placement and occupancy filling.

For the current sweeps, we used `medium.wires.txt`, eight threads, $P = 0.1$, five iterations, and batch size one. Each computation time is the median of three runs per configuration, with one knob varied at a time. All supplied sweep outputs passed validation and had maximum occupancy three.

Table 2 collects the three sweeps. Bold settings are the selected choices; differences within each sweep were small.

We kept static scheduling, which used about 2% less computation time than dynamic. Static avoids dynamic work-dispatch overhead, but this modest difference does not isolate overhead as the cause. Chunk one had the lowest median, about 1.6% below chunk eight; the full chunk sweep spanned only about 1.8%. These results support a provisional configuration, not a claim that scheduling or chunk size dominates performance.

Fine granularity. In `schedule(static, 1)`, a chunk is one candidate route, not one cell. Candidates are assigned round-robin without a dynamic task queue. A candidate still performs a complete path evaluation, so a small chunk does not imply that each scheduled unit contains only a trivial amount of work.

Sweep	Setting	Compute median (s)	Cost
Schedule	static	1.504505	267645
	dynamic	1.535503	267681
Chunk size	1	1.486193	267735
	4	1.513441	267641
	8	1.509649	267645
	16	1.499131	267423
	32	1.502043	267425
Threshold multiplier	2	1.517222	267735
	4	1.486254	267735
	8	1.488452*	267735
	16	1.489102	267735
	32	1.506372	267735

Table 2: Medium-input tuning on GHC at eight threads. Schedule sweep: chunk 8, multiplier 4. Chunk sweep: static, multiplier 4. Threshold sweep: static, chunk 1. All supplied outputs passed validation. *The multiplier-8 time is a supplied correction to the earlier recorded value, not a median recomputed from raw runs.

Importantly, all generated candidates for a particular wire have the same Manhattan length, $d_x + d_y$, and visit the same number of cells. Path lengths vary between wires, but the W-mode parallel loop processes only one wire’s candidates at a time. Therefore, the chunk-one result cannot be explained by balancing different wire lengths. Candidate orientations, bend counts, and memory access patterns can differ even with equal path length; a fine cyclic assignment may distribute those differences, but these effects were not separately measured. The timing data justify choosing chunk one provisionally, not a general claim that finer granularity is always better.

The parallel threshold is the multiplier times the thread count: the tested values correspond to 16, 32, 64, 128, and 256 candidates at eight threads. We retained multiplier four; the corrected medians for 4, 8, and 16 differed by only about 0.2%. The sequential fallback avoids entering a parallel region for very small searches, but this sweep does not establish an exact optimal cutoff or how frequently each threshold changes the execution path.

Our selected configuration is therefore static scheduling, chunk size one, and threshold multiplier four. These decisions are based on medium-input measurements and must still be checked on few and abundant inputs and in the final scaling evaluation.

4.2 Synchronization and Shared Memory

Threads share both occupancy views, but only read them during candidate scoring, so no occupancy locks are needed in W mode. Multiple threads reading the same cache line do not cause write-induced coherence invalidations and may benefit from shared cache reuse. This is distinct from false sharing, which requires writes to a shared cache line; our local bests remain private throughout scoring. Actual cache benefits depend on working-set size and scheduling and have not yet been quantified with cache-miss measurements.

The worksharing loop has an implicit barrier. Each worker then publishes its local best to its own result slot, and the parallel-region end synchronizes completion before the main thread reduces those results. Occupancy removal and insertion remain sequential, outside the parallel region. Thus no critical section or atomic update is needed for each candidate comparison.

We moved the per-thread result-vector allocations outside the iteration and wire loops, reusing storage across searches. Before each parallel search, cost slots reset to the current wire’s starting best; active threads overwrite their route slots. Unused slots cannot select stale routes because the final reduction accepts only strictly cheaper costs. Buffer reuse showed no measurable computation-time improvement, so we retain it as a simple allocation cleanup rather than attributing a speedup to it.

Equal-cost route choices can differ across thread counts and schedules because the reduction does not impose a deterministic candidate-index tie-break. Those choices can affect later wires and explain different final costs without an occupancy mismatch. Consequently, some tuning runs follow different routing trajectories rather than exactly identical work.

4.3 Scaling Limits

Table 3 shows the initial current-implementation medium snapshot, using static chunk eight and multiplier four. This is a preliminary checkpoint, not the final evaluation of the selected chunk-one configuration.

Eight-thread computation speedup was $6.31\times$, with approximately 79% efficiency. The gains are substantial but diminish as threads are added.

We do not expect perfect speedup. Initial route placement, occupancy updates, random-route choices, and the final per-wire reduction remain sequential. Many searches also enter separate parallel regions and barriers. Small searches offer limited candidate-level parallelism, and occupancy reads can be constrained by cache behavior or memory bandwidth. Candidate

Threads	Compute (s)	Compute speedup	Total speedup	Efficiency
1	9.511255	1.00×	1.00×	100.0%
2	4.941826	1.92×	1.92×	96.2%
4	2.664504	3.57×	3.53×	89.2%
8	1.506321	6.31×	6.17×	78.9%

Table 3: Initial medium-input scalability medians on GHC with $P = 0.1$, five iterations, and batch size one. Speedups use this implementation’s one-thread baseline; efficiency uses computation speedup. Total time is initialization plus computation. All outputs passed validation.

generation is now parallel on the parallel path, so sequential full-vector generation is no longer a remaining bottleneck in that path. Profiling is needed to distinguish the remaining synchronization, memory, and sequential-work costs.

We observed improving performance, not a drop-off, from one through eight threads in the initial snapshot. Efficiency declined as thread count increased. At higher counts, limited candidates per wire, synchronization overhead, and memory-system pressure may flatten or reverse scaling, but this remains a hypothesis to test in the PSC experiments.

5 Across-Wires Parallelism

Our initial A-mode baseline assigns batches of wires to workers using dynamic scheduling. Each worker explores its batch’s wires sequentially, remembers all chosen routes, and only then commits the batch’s removals and insertions. One parallel region spans all iterations, with a barrier between iterations. Workers have private random generators and reusable batch-route storage.

Unlike W mode, A cannot remove a wire before its batch finishes selecting routes. The initial scorer therefore read each cell atomically and subtracted one when `cell_on_wire()` found that cell on the current route: $(value - onOldRoute + 1)^2$. That helper scanned up to four old segments for every cell of every candidate. Atomic increment/decrement updates protect both occupancy views; atomic reads permit concurrent scoring without a C++ data race. They do not provide a consistent whole-route snapshot, which the assignment does not require.

5.1 Isolating the Initial Scoring Overhead

The first reported one-thread few-input A computation time was approximately 15 seconds, compared with W’s 2.82 seconds on the same input. This was an exploratory symptom rather than a controlled median-of-three benchmark. Because the discrepancy occurred at one thread, cross-worker update contention and work imbalance could not explain it. Extracting the mode implementations into separate functions improved organization but did not remove the per-cell membership checks; we therefore investigated the scoring loop before tuning scheduling or batch size.

We used a temporary local scoring-only test over the 2,977,130 candidates from the initial few-input wire configuration. It compared W-style scoring, the same scorer with atomic reads, and A’s atomic reads plus membership checks. All variants used equivalent effective occupancy and produced matching cost checksums. Each candidate was scored fully. The test used optimized local Clang compilation and medians of three runs, without an OpenMP worker team. It excluded rerouting iterations, random choices, commit time, and parallel scheduling, so these are not GHC full-program performance results.

Scoring variant	Local median (s)
W-style scoring	0.159
W-style scoring with atomic reads	0.467
Initial A: atomic reads and per-cell membership checks	1.63
Updated A: atomic reads and segment overlap ranges	0.428

Table 4: Scoring-only diagnostic on identical initial few-input candidates. The first three variants isolate the original overhead; the last row is a separate three-run check after the change. All cost checksums matched. These timings are not assignment benchmark times.

Table 4 showed overhead both from the atomic-read variant and from adding per-cell geometric checks. The atomic comparison includes compiler optimization effects, not just the cost of a hardware atomic instruction. The test supported targeting the inner-loop work, without establishing how much of the reported GHC time each factor contributed.

5.2 Moving Geometric Checks Out of the Cell Loop

We now determine overlaps between each candidate segment and the old wire before visiting that segment’s cells. Parallel segments yield overlap intervals;

perpendicular crossings yield single-cell intervals. These ranges are clipped to the candidate’s half-open traversal bounds, sorted, and merged so shared bends are not subtracted twice.

The scorer then walks non-overlapping spans using $(value + 1)^2$ and overlapping spans using $value^2$, where removing the old wire and adding the candidate cancel. This replaces a segment scan at every cell with a small fixed-size range calculation per segment. Every cell is still read atomically and evaluated once, and the final endpoint is included separately. Only geometric overlap metadata is computed: there are no cached costs, prefix sums, skipped candidates, or early termination.

The updated scorer’s local median was 0.428 seconds versus 1.63 seconds for the initial A scorer, approximately a $3.80\times$ scoring-only improvement across the two measurement sets. This is encouraging but not a prediction of the full-program speedup. Cost-reference tests, before/after output comparisons, and atomic-update stress checks passed; a matching GHC before/after benchmark and full OpenMP validation remain to be collected. Batch selection/update timing, random selection, scheduling, and W mode are unchanged.

6 Optimization Hypotheses

The main performance themes we plan to evaluate are workload balance, scheduling overhead, locality, and contention. Within-wires parallelism exposes candidate routes as independent tasks; this should work best for wires with large bounding boxes and many possible routes. Across-wires parallelism exposes wires or batches of wires as tasks; this may require dynamic scheduling or longest-task-first ordering to avoid stragglers.

We will also consider locality-aware across-wires batching as a later experiment. For example, wires could be grouped by bounding-box midpoint or quadrant so that threads tend to work on nearby occupancy cells. This may improve cache locality, but it may also increase contention because nearby wires are more likely to overlap. We will only pursue this direction if cache miss or synchronization measurements suggest that the extra work is justified.

7 Tentative GHC Within-Wires Results

Tables 5–7 record the supplied W-mode outputs for the three required timing inputs. Each input uses a 2048×2048 grid, $P = 0.1$, five iterations, and batch size one. These tables are tentative: repetition counts and compiled scheduling, chunk-size, and threshold settings remain to be confirmed before finalizing the experimental configuration. We do not label these supplied outputs as three-run medians without that confirmation.

Computation time includes initial route placement and occupancy filling. Total time is initialization plus computation. For each input separately, speedups use that input’s own one-thread baseline:

$$S_p^{\text{compute}} = \frac{C_1}{C_p}, \quad S_p^{\text{total}} = \frac{I_1 + C_1}{I_p + C_p}.$$

Times are displayed to three significant figures; speedups are calculated from the unrounded supplied values, not the rounded table entries.

Threads	Init (s)	Compute (s)	Total (s)	Compute S_p	Total S_p	Max occ.	Cost
1	0.0390	2.82	2.86	1.00×	1.00×	2	29240
2	0.0439	1.54	1.58	1.83×	1.81×	2	29240
4	0.0411	0.889	0.930	3.17×	3.08×	2	29240
8	0.0439	0.555	0.599	5.08×	4.77×	2	29240

Table 5: Tentative GHC within-wires results for `few_wires.txt`, 128 wires. All supplied outputs passed validation.

Threads	Init (s)	Compute (s)	Total (s)	Compute S_p	Total S_p	Max occ.	Cost
1	0.0456	9.34	9.39	1.00×	1.00×	3	267723
2	0.0460	4.87	4.91	1.92×	1.91×	3	267957
4	0.0456	2.62	2.66	3.57×	3.52×	3	267771
8	0.0460	1.48	1.52	6.32×	6.16×	3	267735

Table 6: Tentative GHC within-wires results for `medium_wires.txt`, 2048 wires. All supplied outputs passed validation.

At eight threads, computation speedup is $5.08\times$ for few, $6.32\times$ for medium, and $6.48\times$ for abundant wires. Total-time speedups are lower: $4.77\times$, $6.16\times$, and $6.39\times$, respectively. Initialization is a larger fraction of the short few-input run, so it reduces total speedup more noticeably there. The larger

Threads	Init (s)	Compute (s)	Total (s)	Compute S_p	Total S_p	Max occ.	Cost
1*	0.0616	24.3	24.4	1.00×	1.00×	4	1539328
2	0.0625	12.6	12.7	1.93×	1.92×	4	1539380
4	0.0621	6.76	6.83	3.59×	3.57×	4	1538772
8	0.0614	3.75	3.81	6.48×	6.39×	4	1539104

Table 7: Tentative GHC within-wires results for `abundant_wires.txt`, 16384 wires. All supplied outputs passed validation. *The first run’s thread-count line was omitted; one thread is inferred from the supplied order and needs confirmation.

inputs show better measured computation scaling, consistent with more useful work amortizing overhead, although profiling is needed to separate that explanation from memory-system and workload effects.

All three inputs improve monotonically from one through eight threads, with no observed performance drop-off. Few retains cost 29240 at every thread count; medium and abundant costs vary slightly, consistent with the schedule-dependent equal-cost choices discussed earlier. These tables do not replace the required speedup graphs, cache-miss measurements, routing images, or PSC evaluation.

8 Final Implementation Decisions and PSC Evaluation

This section records the final implementation used for the completed PSC benchmark set. It is intentionally detailed so that the design can be audited against the assignment rules and so that individual optimizations can be separated from algorithmic changes. The benchmark source commit was 70c112e; later experiments with a single-thread A-mode fast path and guided A-mode scheduling were reverted because neither provided a robust final improvement. Thus the final routing code matches the code used for the PSC measurements below.

8.1 Final decisions, rationale, and synchronization

Exhaustive route set and direct construction. For every greedy reroute, both modes enumerate the complete assignment-defined set of legal routes with at most three bends. We do not sample a subset, cache route costs, use

prefix sums, or terminate a score early. The optimization is only representational: the old implementation materialized all candidate `Wire` objects in a vector, while the final implementation maps a route index directly to its keypoints and immediately scores it. This reduces allocation and temporary storage traffic without changing the route set or the score of any candidate. In W mode the route index is the unit of parallel work; in A mode one worker generates and scores routes serially for each wire in its assigned batch.

Two occupancy views for spatial locality. The row-major matrix `occupancy[y][x]` makes a horizontal segment scan contiguous. A transposed matrix `occupancy_columns[x][y]` makes the corresponding vertical scan contiguous. Every route insertion/removal updates both views, so they represent identical logical occupancy. This deliberately trades a small amount of write work and memory for locality in the dominant operation: the many full candidate-score scans. It is not a range-query data structure; every candidate still reads every one of its cells. The decision was motivated by profiling and the observation that the scorer dominated runtime, while the transpose makes the vertical case match the horizontal case’s unit-stride access pattern.

W-mode parallel candidate search. Within-wires parallelism preserves the original dependency order across wires: the program removes one wire, evaluates all of that wire’s candidates in parallel against a read-only occupancy state, selects one best route, commits it, and only then advances to the next wire. The per-wire candidate loop uses `schedule(static,1)`. Each worker retains a private best route/cost, then the thread that resumes after the parallel region performs a reduction. The implicit barrier at the end of the worksharing loop and the end of the parallel region guarantee that every local best is complete before the reduction and subsequent occupancy update. Occupancy updates are therefore sequential in W mode; no atomic operations or locks are required during scoring.

Static scheduling, chunk size one, and a candidate-count threshold of four times the requested thread count were selected from medium-input GHC sweeps. The threshold keeps small searches sequential, avoiding the cost of forming a parallel team when there is too little candidate work to amortize it. Static chunk-one scheduling was slightly faster than dynamic scheduling in the sweep; the differences were modest, so we describe this as a measured configuration choice rather than a general scheduling law. The PSC curves later show the expected limit of this approach: there is only one

wire’s candidate set available at a time, so high core counts can run out of useful work or pay disproportionately for barriers and reductions.

A-mode batch selection and atomic commit. Across-wires parallelism exposes batches of wires as the independent units. One OpenMP parallel region spans all simulated-annealing iterations. In each iteration, `schedule(dynamic,1)` assigns a batch to an available worker. The worker scores every wire in the batch, keeps route changes in a private `RouteUpdate` vector, and commits its changes only after all wires in that batch have chosen their routes. Each commit atomically decrements the old route and atomically increments the new route in both occupancy views. Atomic loads are used while scoring, so an overlapping commit cannot create a C++ data race with a reader. The implicit barrier at the end of the `omp for` ensures every commit in an iteration has finished before the next iteration begins.

This design deliberately permits the staleness specified by the assignment: a candidate may have been scored before another batch commits. It introduces no additional staleness beyond the chosen batch structure, and it preserves the final occupancy invariant because every changed route is removed and inserted exactly once. The validator checks this invariant after every benchmark run. Dynamic one-batch scheduling was retained because wires have heterogeneous bounding boxes and hence heterogeneous candidate counts. A guided-scheduling experiment on GHC gave a medium-input eight-thread median of 1.283 s versus an earlier dynamic median of 1.290 s, a difference too small to distinguish from run-to-run variation; abundant-input guided trials were also noisy. The experiment was reverted rather than treating a sub-percent difference as a reliable optimization.

A-mode old-route overlap metadata. Leaving the old wire in A-mode occupancy makes naive scoring expensive if every candidate cell asks whether it lies on the old route. The final scorer first normalizes the old wire into at most four segments. For each candidate segment, it computes a constant-size list of parallel overlaps and perpendicular crossings, clips those intervals to the candidate, sorts and merges them, then walks each cell exactly once. This moves geometric membership checks out of the inner cell loop. It does not omit route cells or reuse scores. A scoring-only diagnostic reduced the original atomic-plus-membership-check case from 1.63 s to 0.428 s on the measured local workload, motivating this optimization. Later GHC profiles still attributed approximately 97% of A-mode cycles to the final candidate-score function, so score computation remains the appropriate explanation

for the remaining A-mode cost rather than OpenMP scheduling.

Rejected single-thread A fast path. We temporarily removed A-mode atomic occupancy operations when only one thread was requested. It preserved validation and output cost, but changed aggregate cycles by only about 1.5% in a `GHC perf stat` comparison. The added code complexity was not justified, so the experiment was reverted. This result also indicates that the remaining serial A-mode gap is not primarily an uncontended atomic-operation problem.

8.2 PSC methodology

We ran a single complete experiment set on a dedicated Bridges-2 Regular Memory node, not a shared or login node. The Slurm job rebuilt the program with `make -B` on the allocated node and used `OMP_PLACES=cores` and `OMP_PROC_BIND=close`. It measured all three timing inputs, both parallel modes, and 1, 2, 4, 8, 16, 32, 64, and 128 threads. Each configuration used $P = 0.1$, five annealing iterations, and $B = 1$, and was repeated three times. The complete matrix contains three inputs, two modes, eight thread counts, and three trials, for 144 runs. All 144 runs printed `Validate Passed: no mismatches`. The raw trials, per-trial total times, and derived medians are preserved in `PSC_BENCHMARK_48936839_RAW.csv` and `PSC_BENCHMARK_48936839_SUMMARY.csv`.

For each input/mode/thread-count point, we report the median of three trials. Computation time is the router’s reported computation interval. Total time for one trial is initialization plus computation, and total-speedup uses the median of those per-trial totals. This is preferable to adding independently rounded medians. Each speedup uses the corresponding one-thread result for the same input and mode:

$$S_p^{\text{compute}} = \frac{C_1}{C_p}, \quad S_p^{\text{total}} = \frac{T_1}{T_p}.$$

8.3 PSC median computation times

Table 8 lists the median computation time in seconds. It is the source table for the computation-speedup plots to be added in the final edit. The results make the workload distinction clear: A-mode exposes many wires to the machine at once and keeps scaling on the medium and abundant inputs, whereas W-mode has only one current wire’s candidate set available.

Threads	Few W	Few A	Medium W	Medium A	Abundant W	Abundant A
1	2.833	4.303	10.649	15.558	29.008	42.591
2	1.435	1.968	5.438	7.436	15.086	20.454
4	0.714	0.973	2.722	3.637	7.728	10.063
8	0.363	0.509	1.460	1.818	4.511	5.026
16	0.195	0.291	0.857	0.906	3.025	2.521
32	0.112	0.234	0.586	0.466	2.460	1.279
64	0.077	0.245	0.513	0.273	2.572	0.715
128	0.074	0.243	0.626	0.161	3.987	0.396

Table 8: PSC Bridges-2 RM median computation times (s), three trials per configuration. Every run passed validation.

8.4 PSC computation and total speedup

Threads	Few W	Few A	Medium W	Medium A	Abundant W	Abundant A
1	1.00	1.00	1.00	1.00	1.00	1.00
2	1.97	2.19	1.96	2.09	1.92	2.08
4	3.97	4.42	3.91	4.28	3.75	4.23
8	7.81	8.45	7.30	8.56	6.43	8.47
16	14.55	14.80	12.43	17.17	9.59	16.89
32	25.20	18.35	18.16	33.36	11.79	33.30
64	36.64	17.59	20.77	57.08	11.28	59.59
128	38.37	17.70	17.01	96.36	7.27	107.60

Table 9: PSC computation speedup. Each column uses its own one-thread median as the baseline.

Threads	Few W	Few A	Medium W	Medium A	Abundant W	Abundant A
1	1.00	1.00	1.00	1.00	1.00	1.00
2	1.97	2.18	1.96	2.09	1.92	2.08
4	3.93	4.38	3.90	4.27	3.75	4.23
8	7.59	8.26	7.25	8.51	6.41	8.45
16	13.72	14.18	12.26	16.94	9.54	16.79
32	22.67	17.39	17.77	32.40	11.72	32.84
64	31.31	16.70	20.25	54.16	11.20	58.07
128	31.60	16.62	16.55	86.72	7.24	101.84

Table 10: PSC total speedup. Initialization is serial and becomes visible once computation is very short, especially for the few-input cases.

8.5 Interpretation and comparison with the reference

At eight threads, both modes scale well for every input: W obtains computation speedups of $7.81\times$, $7.30\times$, and $6.43\times$ for few, medium, and abundant

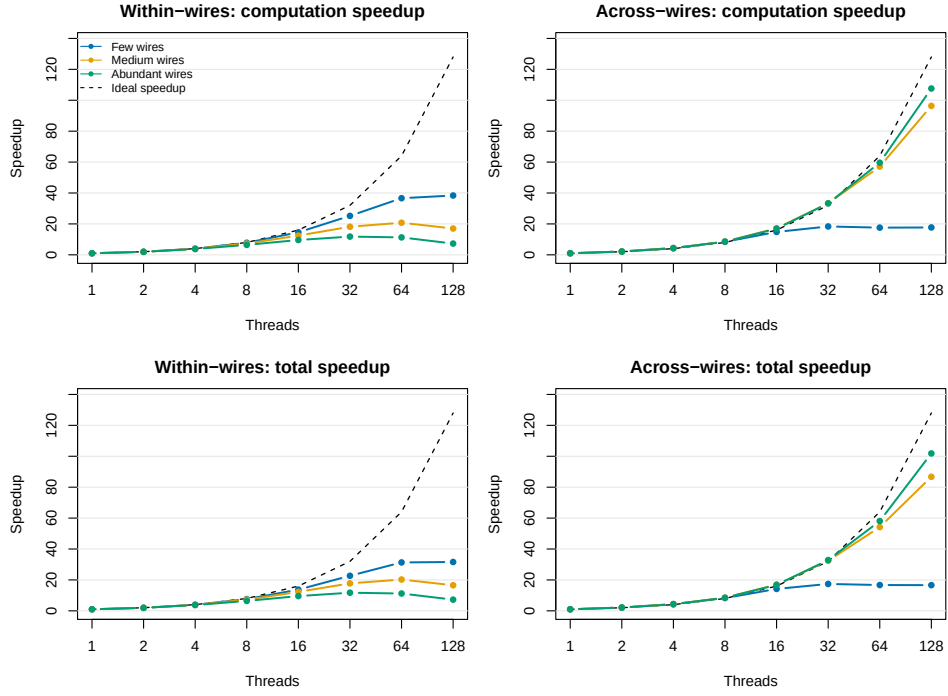


Figure 1: PSC Bridges-2 RM speedup curves derived from the median-of-three summary data. The dashed line is ideal linear speedup. W-mode plateaus when a single wire’s candidate search cannot keep all workers busy; A-mode retains a larger pool of concurrent wires on the medium and abundant inputs.

inputs, while A obtains $8.45\times$, $8.56\times$, and $8.47\times$. The slight superlinear values for A at eight threads should not be treated as evidence that more than eight cores of work occurred; they are consistent with normal run-to-run variation and cache/work-distribution effects relative to the selected one-thread median.

The two strategies diverge at high core counts. W peaks at 64 threads on few and medium, and at 32 threads on abundant, before flattening or regressing. For medium and abundant inputs, candidate searches for individual wires cannot always supply enough balanced work to amortize repeated parallel-region and barrier costs. The abundant W-mode 128-thread trials also had larger variation than the rest of the PSC dataset, so the 128-thread median should be reported with caution rather than over-interpreted.

A-mode instead reaches $96.36\times$ computation speedup on medium and $107.60\times$ on abundant at 128 threads. These results are consistent with A-mode’s larger pool of concurrently selectable wires. Few-input A-mode levels off around 32 threads because only 128 wires are available and the fixed initialization cost becomes a larger fraction of total time. The total-speedup table makes that serial-initialization effect explicit: for example, medium A-mode falls from $96.36\times$ computation speedup to $86.72\times$ total speedup at 128 threads.

Table 11 compares our PSC one-thread computation medians with the handout’s good-performance reference. W-mode is close to the reference on few and medium and is slower on abundant. A-mode’s one-thread time is higher than the reference, even though its high-core-count scaling is strong. Speedup therefore must be read together with absolute computation time: a slower one-thread baseline can inflate a relative speedup. This is one reason we report both tables and preserve the raw measurements.

Input	Our W	Ref. W	Our A	Ref. A
Few	2.833	3.138	4.303	3.066
Medium	10.649	10.345	15.558	11.460
Abundant	29.008	24.817	42.591	26.707

Table 11: PSC one-thread computation time (s): our medians versus the handout’s good-performance reference.

9 Remaining Report Work

The PSC data collection is complete and preserved. The remaining work is to generate the required PSC computation- and total-speedup figures from the summary CSV, collect the final controlled GHC tables for both W and A at 1, 2, 4, and 8 threads, and complete the two required A-mode GHC sensitivity studies (random-route probability and problem size). The earlier tentative GHC W-mode tables are retained above as working notes; they should be replaced by median-of-three final measurements before submission. We should also include one or more final route visualizations and prune the exploratory implementation history once the required results and figures are in place.